Libreto Oficial de Sustentación - DocuProcess Cloud (15 Diapositivas en Canva)

Asignatura: Plataformas II - Departamento de TIC - Universidad ICESI
Autores y Expositores: Santiago Gómez Robledo & Valeria Ortiz
Duración Total: 15 Minutos (10 minutos exposición teórica/arquitectónica + 4 minutos demostración en vivo + 1 minuto conclusiones y preguntas)
Entorno de Demostración: Kubernetes v1.37.0 (Minikube v1.39.0 sobre Containerd 2.3.4, 8 vCPUs, 14 GiB RAM)

Estructura Temporal y Distribución de Roles (15 Diapositivas)

Diapositiva	Título de la Diapositiva	Orador Principal	Tiempo Sugerido
1	Portada & Presentación Ejec...	Santiago & Valeria	00:00 - 00:50
2	El Desafío Enterprise y la Op...	Valeria Ortiz	00:50 - 01:45
3	Propuesta de Valor y Filosofí...	Valeria Ortiz	01:45 - 02:40
4	Topología Kubernetes y Seg...	Santiago Gómez Robledo	02:40 - 03:40
5	Capa de Aplicación: Frontend...	Santiago Gómez Robledo	03:40 - 04:35
6	Persistencia Transaccional y ...	Santiago Gómez Robledo	04:35 - 05:30
7	Seguridad Perimetral L3/L4: ...	Santiago Gómez Robledo	05:30 - 06:30
8	Identidad Enterprise Federad...	Valeria Ortiz	06:30 - 07:30
9	Observabilidad de Aplicación:...	Valeria Ortiz	07:30 - 08:25
10	Observabilidad de Infraestruc...	Valeria Ortiz	08:25 - 09:15
11	Trazabilidad Distribuida E2E y...	Valeria Ortiz	09:15 - 10:10
12	Elasticidad Automática con H...	Santiago Gómez Robledo	10:10 - 11:05
13	Gobierno Financiero y FinOp...	Valeria Ortiz	11:05 - 12:00
14	Despliegue Progresivo: Estrat...	Santiago Gómez Robledo	12:00 - 13:00
15	Validación Operativa (Live De...	Santiago & Valeria	13:00 - 15:00

Desarrollo Diapositiva por Diapositiva

Diapositiva 1: Portada & Presentación Ejecutiva

- **Visual en Canva:** Logotipo estilizado de DocuProcess Cloud, título de la plataforma, curso *Plataformas II - Universidad ICESI*, nombres de los integrantes e insignias de tecnologías (*Kubernetes, Spring Boot, React 19, Keycloak, Prometheus, Grafana, ELK, Kubecost*).
- **Tiempo:** 50 segundos (00:00 - 00:50).
- **Oradores:** Santiago Gómez Robledo y Valeria Ortiz.

Santiago:

*"Buenas tardes profesor y compañeros. Hoy les presentamos **DocuProcess Cloud**, nuestro proyecto final para el curso de **Plataformas II** en la Universidad ICESI. Mi nombre es Santiago Gómez Robledo y junto a mi compañera Valeria Ortiz diseñamos una plataforma SaaS Enterprise orientada a la ingesta, procesamiento asíncrono y análisis inteligente de documentos e imágenes corporativas."*

> **Valeria:**
> *"En este proyecto no nos limitamos a escribir código de backend y frontend; implementamos una arquitectura integral cloud-native operada sobre Kubernetes que resuelve los desafíos de las empresas modernas: autenticación federada con SSO y MFA, aislamiento perimetral estricto Zero-Trust, observabilidad multinivel bajo los modelos RED y USE, gobernanza financiera con FinOps y despliegues progresivos sin indisponibilidad mediante Canary Deployments."*

Diapositiva 2: El Desafío Enterprise y la Oportunidad SaaS

- **Visual en Canva:** Esquema del dolor del cliente: diagrama de servidor monolítico colapsando ante avalancha de peticiones HTTP síncronas en cierre de mes, timeouts en navegador y error HTTP 504 Gateway Timeout.
- **Tiempo:** 55 segundos (00:50 - 01:45).
- **Orador:** Valeria Ortiz.

> **Valeria:**
> *"Para dimensionar la relevancia de la arquitectura, examinemos el problema de negocio. En organizaciones medianas y grandes - como bancos, aseguradoras y firmas legales - los cierres de mes generan avalanchas de miles de facturas y contratos pesados que deben digitalizarse al mismo tiempo.*
>
> *En los sistemas tradicionales con llamadas HTTP síncronas, el servidor web retiene el hilo de conexión del cliente mientras ejecuta la extracción OCR pesada. El resultado es fatal: hilos agotados, navegadores congelados, errores HTTP 504 Gateway Timeout y pérdidas millonarias por retrasos operativos.*
>
> *Comprendimos que la solución no era comprar servidores más grandes, sino desacoplar radicalmente la ingesta del cómputo pesado mediante un pipeline cloud-native orientado a eventos."*

Diapositiva 3: Propuesta de Valor y Filosofía Cloud-Native

- **Visual en Canva:** Flujo desacoplado: Ingesta en <50ms retornando `HTTP 202 Accepted` + `tracking_id`, almacenamiento seguro en PVC y pools elásticos de workers en segundo plano.
- **Tiempo:** 55 segundos (01:45 - 02:40).
- **Orador:** Valeria Ortiz.

> **Valeria:**
> *"Nuestra propuesta de valor se basa en tres principios de ingeniería:*
> *1. **Ingesta Inmediata No Bloqueante:** Cuando un usuario o un sistema ERP sube un lote de documentos, el servicio valida la estructura en menos de 50 milisegundos, almacena el binario y retorna de inmediato un código **HTTP 202 Accepted** junto a un UUID de seguimiento (`tracking_id`).*
> *2. **Cómputo Elástico en Background:** El procesamiento documental pesado se despacha hacia pools de trabajadores elásticos que extraen metadatos y notifican el resultado sin bloquear al usuario.*

*3. **Arquitectura Multi-Tenant:** Cada cliente corporativo opera con aislamiento estricto de recursos y alta disponibilidad garantizada las 24 horas del día."*

Diapositiva 4: Topología Kubernetes y Segmentación de Namespaces

- **Visual en Canva:** Diagrama del clúster Minikube distribuido en 6 namespaces (`docuprocess-app`, `data`, `security`, `logging`, `monitoring`, `finops`), ilustrando la separación nítida entre el Plano de Control y el Plano de Datos.
- **Tiempo:** 1 minuto (02:40 - 03:40).
- **Orador:** Santiago Gómez Robledo.

Santiago:
*"Para alojar este ecosistema, configuramos un clúster de Kubernetes v1.32.0 sobre Minikube con 8 núcleos de CPU y 14 GiB de memoria RAM utilizando el runtime Containerd. Una restricción arquitectónica central fue **no utilizar Terraform bajo ninguna circunstancia**, basando el 100% del despliegue en manifiestos declarativos nativos de Kubernetes.*

Segmentamos la solución en 6 namespaces de propósito específico:
- `docuprocess-app`: Cargas de trabajo de usuario (Frontend React 19 y Backend Java 21).
- `docuprocess-data`: PostgreSQL 17 relacional con almacenamiento dinámico PVC.
- `docuprocess-security`: Keycloak 26 gestionando realms y políticas OIDC.
- `docuprocess-logging`: Stack ELK (Elasticsearch 8.17, Logstash y Kibana).
- `docuprocess-monitoring`: Prometheus, Alertmanager, OpenTelemetry y Jaeger.
- `docuprocess-finops`: Kubecost 2.8.7 para monitoreo de gasto.

Todo el acceso externo se centraliza a través del Ingress Controller NGINX, terminando conexiones y aplicando balanceo L7."

Diapositiva 5: Capa de Aplicación: Frontend React 19 y Backend Java 21

- **Visual en Canva:** Diagrama de Clean Architecture en 4 capas (Domain, Application, Infrastructure, Presentation) junto a las insignias de React 19, TypeScript, Vite, Tailwind CSS y Java 21 Spring Boot 3.5.
- **Tiempo:** 55 segundos (03:40 - 04:35).
- **Orador:** Santiago Gómez Robledo.

Santiago:
*"En el plano de aplicación aplicamos los principios de **Clean Architecture**, aislando completamente las reglas del dominio de los frameworks web y bases de datos:*

*- **Frontend:** Desarrollado en React 19 con TypeScript, Vite y Tailwind CSS. Implementa Dark Mode nativo, diseño responsive para dispositivos móviles y comunicación asíncrona mediante tokens federados OIDC.*
*- **Backend:** Construido en Java 21 LTS y Spring Boot 3.5.x. Está empaquetado en imágenes de contenedor OCI multi-stage seguras, que ejecutan con un usuario sin privilegios (`UID 10001 non-root`).*
*- **Instrumentación Nativa:** El backend incorpora Spring Boot Actuator para exportar métricas a Prometheus y el SDK de OpenTelemetry para inyectar cabeceras de trazabilidad en cada solicitud."*

Diapositiva 6: Persistencia Transaccional y StatefulSets

- **Visual en Canva:** Diagrama de PostgreSQL 17 StatefulSet, volúmenes persistentes PVCs (`postgres-data-pvc` 5Gi y `docuprocess-storage-pvc` 10Gi) y esquemas desacoplados `docuprocess_db` y `keycloak_db`.
- **Tiempo:** 55 segundos (04:35 - 05:30).
- **Orador:** Santiago Gómez Robledo.

> **Santiago:**
> *"Para garantizar la integridad y durabilidad de los datos corporativos, desplegamos **PostgreSQL 17** mediante un StatefulSet de Kubernetes:*
>
> *- **Almacenamiento Persistente Dinámico:** Vinculamos el PVC `postgres-data-pvc` de 5 GiB bajo la clase `standard`, lo que garantiza que ante reinicios o fallas del pod, los datos jamás se pierdan.*
> *- **Separación de Esquemas:** Aislamos la base de datos de negocio (`docuprocess_db`) de la base de datos de identidad (`keycloak_db`), impidiendo que un problema de transacciones en la aplicación comprometa la autenticación.*
> *- **Criptografía y Auditoría:** Habilitamos las extensiones `pgcrypto` y `uuid-ossp` directamente en el motor de base de datos.*
> *- **Almacenamiento de Binarios:** Un segundo PVC de 10 GiB (`docuprocess-storage-pvc`) actúa como repositorio centralizado para los documentos procesados."*

Diapositiva 7: Seguridad Perimetral L3/L4: Modelo Zero-Trust

- **Visual en Canva:** Diagrama de red de Capa 3 y 4 (OSI) ilustrando las NetworkPolicies, la regla predeterminada `Default Deny` y el bloqueo de cualquier tráfico lateral hacia PostgreSQL.
- **Tiempo:** 1 minuto (05:30 - 06:30).
- **Orador:** Santiago Gómez Robledo.

> **Santiago:**
> *"En un SaaS Enterprise multi-tenant, la seguridad perimetral no puede asumir que la red interna es segura. Por ello implementamos un modelo **Zero-Trust** con Kubernetes NetworkPolicies nativas en las Capas 3 y 4 del modelo OSI:*
>
> *- **Default Deny:** Todo el tráfico entre pods se bloquea de forma predeterminada.*
> *- **Ingreso Controlado:** El Frontend y el Backend únicamente aceptan conexiones HTTP procedentes de los pods del namespace `ingress-nginx`.*
> *- **Protección de Base de Datos:** PostgreSQL en el puerto TCP 5432 rechaza cualquier paquete que no provenga estrictamente de pods etiquetados `app: docuprocess-backend` o `app.kubernetes.io/name: keycloak`.*
>
> *Cualquier intento de escaneo lateral por parte de un contenedor comprometido es descartado a nivel de kernel mediante el CNI sin responder paquetes SYN-ACK."*

Diapositiva 8: Identidad Enterprise Federada y SSO (Keycloak 26)

- **Visual en Canva:** Pantalla de Login SSO corporativo, flujo OIDC con OpenID Connect y Authorization Code Flow con PKCE, y detalle de roles RBAC (`TENANT_ADMIN`, `OPERATOR`, `AUDITOR`) y MFA

(TOTP).

- **Tiempo:** 1 minuto (06:30 - 07:30).
- **Orador:** Valeria Ortiz.

> **Valeria:**
> *"En la capa de seguridad e identidad integramos **Keycloak 26**, la plataforma de identidad federada de código abierto basada en Quarkus:*
>
> *- **Estándar Abierto:** Implementamos OpenID Connect (OIDC) y OAuth 2.0 con flujo Authorization Code con PKCE para el Frontend React.*
> *- **Control de Acceso Basado en Roles (RBAC):** Definimos roles estrictos en el realm `docuprocess-enterprise`: `TENANT_ADMIN` para administración de la organización, `DOCUMENT_OPERATOR` para la carga y gestión de archivos, y `AUDITOR` con acceso exclusivo de lectura a métricas y logs.*
> *- **Defensa Activa:** Incorporamos protección contra ataques de fuerza bruta (bloqueo automático tras intentos erróneos) y **Autenticación Multifactor (MFA)** obligatoria mediante códigos TOTP temporales.*
> *- **Tokens JWT:** Los tokens de sesión son validados criptográficamente en memoria por el backend Java sin saturar la base de datos."*

Diapositiva 9: Observabilidad de Aplicación: Modelo RED

- **Visual en Canva:** Paneles del Modelo RED en Grafana mostrando Rate (RPS por versión), Errors (tasa 5xx < 2% con Alertmanager) y Duration (latencia P95/P99 < 2.5s).
- **Tiempo:** 55 segundos (07:30 - 08:25).
- **Orador:** Valeria Ortiz.

> **Valeria:**
> *"Pasando a la observabilidad, estructuramos el monitoreo bajo dos marcos reconocidos internacionalmente. El primero es el **Modelo RED**, orientado al cliente y la aplicación:*
>
> *- **Rate (Tasa de Peticiones):** Medimos las solicitudes por segundo desglosadas por versión de microservicio (Stable vs Canary).*
> *- **Errors (Tasa de Errores):** Supervisamos el porcentaje de respuestas HTTP 4xx y 5xx. Configuramos una regla de alerta crítica en Alertmanager que se dispara si la tasa de errores 5xx supera el 2%.*
> *- **Duration (Latencia):** Controlamos el tiempo de respuesta del procesamiento documental mediante percentiles estadísticos y . Si la latencia excede los 2.5 segundos, el sistema emite una alarma inmediata de degradación de servicio."*

Diapositiva 10: Observabilidad de Infraestructura: Modelo USE

- **Visual en Canva:** Paneles del Modelo USE mostrando Utilization (CPU/RAM cAdvisor), Saturation (presión de memoria cgroups) y Errors (restarts de pods y alertas de nodo >85%).
- **Tiempo:** 50 segundos (08:25 - 09:15).
- **Orador:** Valeria Ortiz.

Valeria:
*"En paralelo, implementamos el **Modelo USE**, orientado al hardware y la salud física de los nodos:*

*- **Utilization (Utilización):** Prometheus consulta a cAdvisor cada 15 segundos para registrar el consumo porcentual de CPU, memoria y almacenamiento en cada nodo y pod del clúster.*
*- **Saturation (Saturación):** Monitoreamos las colas de procesamiento del sistema operativo y la presión de memoria en los límites cgroups.*
*- **Errors (Errores de Infraestructura):** Detectamos eventos de reinicios inesperados de contenedores, fallos de montaje de volúmenes persistentes y caídas por falta de memoria (OOMKilled).*

Alertmanager vigila que si la utilización de CPU del nodo supera el 85%, el equipo de infraestructura reciba una notificación proactiva antes de que el usuario lo note."

Diapositiva 11: Trazabilidad Distribuida E2E y Correlación de Logs

- **Visual en Canva:** Diagrama del flujo de telemetría: Logstash (TCP 5000) Elasticsearch 8.17 Kibana 5601 correlacionado con OpenTelemetry Collector (4317/4318) y Jaeger All-in-One (16686).
- **Tiempo:** 55 segundos (09:15 - 10:10).
- **Orador:** Valeria Ortiz.

Valeria:
*"El verdadero valor de la observabilidad reside en la **correlación distribuida**. Un log aislado o una gráfica solitaria no resuelven incidentes complejos.*

Implementamos un pipeline de dos vías:
*1. **Logging Centralizado:** El backend despacha registros estructurados en JSON a través de sockets TCP 5000 hacia Logstash, el cual los enriquece y almacena en índices diarios de Elasticsearch 8.17, visualizables desde Kibana.*
*2. **Trazabilidad Distribuida:** El SDK de OpenTelemetry inyecta la cabecera estándar W3C `traceparent`. Las trazas viajan por gRPC hacia el OpenTelemetry Collector y se almacenan en Jaeger.*

Si un documento falla en Kibana, el analista copia el `traceId` y lo pega en Jaeger: en pantalla aparece el árbol de ejecución completo con el tiempo exacto que tomó el Ingress, el controlador Java y la consulta SQL en PostgreSQL."

Diapositiva 12: Elasticidad Automática con HPA v2

- **Visual en Canva:** Ecuación matemática del Horizontal Pod Autoscaler, gráfica de escalado dinámico de 2 a 8 réplicas y explicación de la ventana de estabilización de 180 segundos.
- **Tiempo:** 55 segundos (10:10 - 11:05).
- **Orador:** Santiago Gómez Robledo.

Santiago:
*"Para que la plataforma soporte cargas impredecibles de procesamiento documental sin desperdiciar recursos computacionales, configuramos el **Horizontal Pod Autoscaler (HPA v2)** conectado al `metrics-server`.*

El HPA evalúa la métrica bajo su rigurosa formulación matemática:

*- **Límites de Operación:** Establecimos un piso de 2 réplicas para alta disponibilidad y un techo elástico de 8 réplicas.*

*- **Umbral de Disparo:** Se activa cuando el promedio de consumo de CPU de los pods supera el 70%.*
*- **Ventana de Estabilización:** Fijamos una ventana de enfriamiento de 180 segundos para el escalado descendente. Esto elimina el fenómeno de 'aleteo' (flapping), evitando que los pods se creen y destruyan constantemente ante picos momentáneos de tráfico."*

Diapositiva 13: Gobierno Financiero y FinOps con Kubecost

- **Visual en Canva:** Captura de Kubecost 2.8.7 mostrando la atribución de costos por Namespace y el cálculo del Unit Economics de $0.0014 USD por documento procesado.
- **Tiempo:** 55 segundos (11:05 - 12:00).
- **Orador:** Valeria Ortiz.

Valeria:
*"En el desarrollo de software SaaS contemporáneo, la excelencia técnica debe ir acompañada de sostenibilidad financiera. Por ello desplegamos **Kubecost 2.8.7** en el namespace* `docuprocess-finops`*:*

*- **Atribución de Costos en Tiempo Real:** Medimos el gasto exacto en dólares de CPU, memoria y almacenamiento persistente (PVC) desglosado por Namespace y Pod.*
*- **Detección de Desperdicio:** Identificamos sobredimensionamiento de recursos para calibrar los requests y limits de cómputo.*
*- **Unit Economics (Economía Unitaria):** Logramos calcular el costo computacional marginal por documento procesado, el cual se sitúa en **$0.0014 USD**. Esto le permite al área financiera de la startup diseñar planes de suscripción con márgenes brutos de ganancia superiores al 75% respaldados matemáticamente por la infraestructura."*

Diapositiva 14: Despliegue Progresivo: Estrategia Canary Deployment

- **Visual en Canva:** Diagrama del NGINX Ingress dividiendo el tráfico 90% hacia `docuprocess-backend-stable` (v1.0.0) y 10% hacia `docuprocess-backend-canary` (v2.0.0), junto con la cabecera `X-Canary-Test: always`.
- **Tiempo:** 1 minuto (12:00 - 13:00).
- **Orador:** Santiago Gómez Robledo.

Santiago:
*"Para actualizar la plataforma sin riesgo de interrupción del negocio, implementamos la estrategia de **Canary Deployment** directamente en el Ingress Controller:*

*- **División Ponderada:** El 90% de las solicitudes se canaliza hacia la versión probada* `v1.0.0` *(Stable), mientras que el 10% se deriva a la nueva versión* `v2.0.0` *(Canary).*
*- **Enrutamiento por Cabecera:** Mediante la cabecera* `X-Canary-Test: always`*, el equipo de QA o usuarios de prueba pueden interactuar de forma determinista con la versión Canary en el entorno de producción sin afectar al resto de los clientes.*
*- **Validación Estadística:** Ejecutamos pruebas con lotes de 100 solicitudes concurrentes confirmando una distribución real del 88% Stable y 12% Canary sin pérdida de paquetes.*
*- **Rollback Inmediato:** Si la versión Canary presenta fallos, el rollback toma exactamente dos segundos: fijamos el peso en 0 en la anotación del Ingress y escalamos el Canary a cero réplicas."*

Diapositiva 15: Validación Operativa (Live Demo) y Conclusiones

- **Visual en Canva:** Esquema de los 6 pasos de la demostración en vivo (Keycloak React 19 ELK Jaeger Grafana HPA DoD 100%) y conclusiones de ingeniería.
- **Tiempo:** 2 minutos (13:00 - 15:00).
- **Oradores:** Santiago Gómez Robledo & Valeria Ortiz.

> **Santiago:**
> *"Para concluir la exposición, demostraremos en vivo la operación de los subsistemas en nuestro clúster Minikube:"*
> *(Abre `http://app.docuprocess.local`)*
> *"Iniciamos sesión en el Frontend React 19 a través de Keycloak SSO con nuestro usuario `srobledo`."*

> **Valeria:**
> *(Ejecuta la ingesta de un documento)*
> *"Subimos una factura corporativa y recibimos instantáneamente el código 202 con su tracking UUID. En Kibana (`http://kibana.docuprocess.local`) vemos el log estructurado con su `traceId`. En Jaeger (`http://jaeger.docuprocess.local`) inspeccionamos la traza distribuida a través de todos los microservicios. Y en Grafana (`http://grafana.docuprocess.local`) confirmamos en tiempo real los tableros de los modelos RED y USE."*

> **Santiago:**
> *(Ejecuta `./scripts/e2e-validate.sh` en la terminal)*
> *"En consola ejecutamos nuestro script de validación integral `./scripts/e2e-validate.sh`: como pueden observar, los 8 criterios de la Definition of Done arrojan un resultado del **100% PASS**.*
>
> *Este proyecto nos demuestra que la ingeniería de plataformas moderna no es solo aprovisionar infraestructura, sino crear un ecosistema observable, seguro y económicamente viable. Agradecemos al profesor y compañeros por su atención en este curso de **Plataformas II**. ¡Muchas gracias y quedamos atentos a sus preguntas!"*

Banco de Preguntas y Respuestas Técnicas para los Evaluadores

P1: ¿Por qué eligieron Minikube con driver Docker en lugar de una máquina virtual o nube pública?

> **Respuesta (Santiago):** *"Minikube v1.39 con el driver Docker nos permite una paridad total con clústeres de producción utilizando el runtime moderno `containerd 2.3.4`. Asignamos 8 vCPUs y 14 GiB de memoria, lo que permite ejecutar simultáneamente 6 namespaces, 25 pods y stacks demandantes como Elasticsearch y Keycloak de forma reproducible, determinista y con cero costo de nube durante la fase de desarrollo y validación académica."*

P2: ¿Cómo garantizan que la versión Canary no corrompa los esquemas de base de datos?

> **Respuesta (Santiago):** *"Aplicamos el principio de cambios de base de datos no destructivos (*

expand-and-contract pattern). La versión Canary `v2.0.0` *y la versión Stable* `v1.0.0` *comparten un esquema en PostgreSQL compatible hacia atrás y hacia adelante, utilizando claves UUID y columnas anulables. Toda migración destructiva (como renombrar o eliminar columnas) solo se ejecuta una vez que la versión Canary alcanza el 100% del tráfico y se retira la versión previa."*

P3: ¿Por qué usar OpenTelemetry Collector si Prometheus ya recopila métricas?

Respuesta (Valeria): *"OpenTelemetry desacopla la instrumentación de las aplicaciones del almacenamiento final. El backend de Spring Boot emite trazas mediante el protocolo estándar OTLP (gRPC/HTTP). El OTel Collector actúa como intermediario: procesa en lotes (batch), limita la memoria y reenvía las trazas a Jaeger. Si mañana la startup decide migrar de Jaeger a Datadog o Grafana Tempo, solo cambiamos una línea en la configuración del Collector sin tener que recompilar ni tocar una sola línea del código Java."*

P4: ¿Qué sucede si el pod de Keycloak se cae? ¿Se bloquea todo el clúster?

Respuesta (Valeria): *"No. En el Frontend, los usuarios con sesión activa retienen su token JWT en memoria durante su ciclo de validez (15 minutos). El backend Spring Boot valida la firma criptográfica del JWT utilizando la clave pública RSA obtenida al inicio (JWKS). Por ende, una caída temporal de Keycloak no interrumpe el procesamiento de documentos en curso; únicamente afectaría a los nuevos inicios de sesión mientras el Deployment de Kubernetes reinicia automáticamente el pod mediante sus probes de salud."*